

PROYECTO INTERMODULAR

PROGRAMACIÓN

Diseño de un juego de rol por turnos en Java (Witcher RPG)

3^{er} Trimestre

Nombre del alumno/a: Aarón Hernández Moreno y Víctor de Lucas Fernández

Curso académico: 1º DAM G2

Tutor del proyecto: Carlos Galarón

ÍNDICE PAGINADO

1. JUSTIFICACIÓN DEL PROYECTO

2. INTRODUCCIÓN

3. OBJETIVOS

- 3.1. Objetivo general
- 3.2. Objetivos específicos
- 3.3. Objetivos funcionales
- 3.4. Objetivos del proyecto del 3er trimestre

4. DESARROLLO

- 4.1. Fundamentación teórica
- 4.2. Materiales y métodos
- 4.3. Resultados y análisis

5. DIAGRAMA DE CLASES

6. CONCLUSIONES

7. LÍNEAS DE INVESTIGACIÓN FUTURAS

8. BIBLIOGRAFÍA



1. JUSTIFICACIÓN DEL PROYECTO

En la asignatura de Programación se planteó desarrollar un juego completo que sirviera para poner en práctica los pilares de la programación orientada a objetos. En lugar de un ejercicio teórico, se optó por un juego de rol por turnos ambientado en el universo de The Witcher, con un sistema de combate entre dos equipos de tres personajes cada uno.

La justificación del proyecto es doble. Por un lado, un juego de combate por turnos es un escenario ideal para aplicar la herencia (personajes, armas, hechizos y estados que comparten atributos pero se comportan de forma distinta), el polimorfismo (cada subclase calcula su daño o su efecto a su manera) y la encapsulación (cada clase protege sus propios datos). Por otro lado, el uso de colecciones (listas y mapas) surge de forma natural: un equipo es una lista de personajes, cada personaje tiene una lista de hechizos y estados, y los cooldowns se gestionan con un mapa.

Además, el proyecto es la base sobre la que después se construye la parte intermodular de Bases de Datos, por lo que un diseño de clases limpio y coherente facilita enormemente la posterior incorporación de la persistencia.

2. INTRODUCCIÓN

Esta memoria documenta el diseño y la implementación en Java del juego Witcher RPG. El juego enfrenta a dos equipos: el bando bueno, formado por Geralt de Rivia, Yennefer de Vengerberg y Ciri; y el bando malo, La Cacería Salvaje, formado por Imlerith, Caranthir y Eredin.

Cada personaje pertenece a una de tres clases (Guerrero, Mago o Sacerdote), cada una con mecánicas propias. Los personajes disponen de armas (cuerpo a cuerpo y a distancia), hechizos (de daño y de curación, directos o en el tiempo) y pueden verse afectados por estados alterados (Quemadura, Veneno y Renovar) que actúan turno a turno.

El combate se desarrolla por rondas, y cada ronda sigue un orden estricto de fases: primero las acciones de todos los personajes vivos, después el procesamiento de los estados activos y, por último, la comprobación de la condición de victoria. A lo largo de la memoria se describe la fundamentación teórica del diseño, las herramientas empleadas y el resultado: la jerarquía de clases, sus relaciones y el motor de combate.

3. OBJETIVOS

3.1. Objetivo general

Diseñar e implementar en Java un juego de rol por turnos orientado a objetos que aplique de forma práctica la herencia, el polimorfismo, la encapsulación y el uso de colecciones, con un motor de combate por rondas entre dos equipos de tres personajes.

3.2. Objetivos específicos

- Diseñar una jerarquía de personajes con una clase abstracta base (Personajes) y tres subclases (Guerrero, Mago, Sacerdote), cada una con atributos y comportamiento propios.
- Modelar las armas mediante una clase abstracta Armas y sus subclases ArmaCuerpoACuerpo y ArmaADistancia, con un método polimórfico de cálculo de daño.
- Modelar los hechizos con una clase abstracta Hechizos y cuatro subtipos (daño directo, daño en el tiempo, curación directa y curación en el tiempo).
- Implementar los estados alterados (Quemadura, Veneno, Renovar) como subclases de una clase abstracta Estados que actúan por turno.
- Construir un motor de combate por rondas con sus tres fases (acciones, procesamiento de estados y comprobación de victoria).
- Gestionar los cooldowns de los hechizos mediante un mapa (Map<String, Integer>) y los equipos, hechizos y estados mediante listas (ArrayList).
- Validar las reglas del juego: la vida nunca baja de 0 ni supera el máximo, y no se puede atacar a personajes muertos.

3.3. Objetivos funcionales

Los siguientes objetivos funcionales, recogidos en el enunciado del proyecto de Programación, se han cumplido totalmente:

Objetivo funcional	Estado
Jerarquía de personajes con herencia (Guerrero, Mago, Sacerdote) y polimorfismo.	✓ Cumplido totalmente
Sistema de armas con cuerpo a cuerpo y a distancia y cálculo de daño.	✓ Cumplido totalmente
Catálogo de hechizos con los cuatro tipos obligatorios	✓ Cumplido totalmente

Objetivo funcional	Estado
(daño/curación, directo/en el tiempo).	
Estados alterados por turno: Quemadura, Veneno y Renovar.	✓ Cumplido totalmente
Lógica de acumulación de estados (si existe se renueva, si no se añade).	✓ Cumplido totalmente
Gestión de cooldowns con un Map y reducción por ronda.	✓ Cumplido totalmente
Bucle de combate por rondas con sus tres fases.	✓ Cumplido totalmente
Validación de objetivos vivos y de los límites de vida y recurso.	✓ Cumplido totalmente
Salida por consola detallada de cada ronda y resumen final.	✓ Cumplido totalmente

3.4. Objetivos del proyecto del 3er trimestre

El proyecto parte del código desarrollado en el segundo trimestre y lo amplía con persistencia en base de datos, mejoras en la lógica del juego y nuevos módulos. Las siguientes características, recogidas en el enunciado del proyecto, se han cumplido totalmente:

Característica del enunciado	Estado
Persistencia de partida: guardar y cargar en BD el estado completo de una partida y restaurarlo fielmente.	✓ Cumplido totalmente
Ranking global: clasificación de jugadores en BD ordenada por criterios y actualizada automáticamente.	✓ Cumplido totalmente
Historial de partidas: registro en BD de las acciones relevantes de cada combate, consultable después.	✓ Cumplido totalmente
Logros: sistema de logros persistente con condiciones de desbloqueo y sin duplicidades.	✓ Cumplido totalmente
Guardado automático: almacenamiento del estado en momentos clave (tras cada turno) garantizando la integridad.	✓ Cumplido totalmente
Dificultad configurable: valores almacenados en BD que modifican el comportamiento del juego sin tocar el código.	X No realizado
Corrección de lo comentado en clase: refactorizaciones, separación de responsabilidades y buenas prácticas.	✓ Cumplido totalmente
Tests con JUnit (obligatorio): pruebas unitarias de la lógica de negocio con casos normales, límite y de error.	✓ Cumplido totalmente
Visualización de datos con XChart (punto extra): gráficos integrados mediante Maven en el pom.xml.	✓ Cumplido totalmente

4. DESARROLLO

4.1. Fundamentación teórica

El diseño se apoya en los cuatro pilares de la programación orientada a objetos, aplicados a las cuatro grandes familias de clases del juego: personajes, armas, hechizos y estados.

Herencia

Cada familia tiene una clase abstracta base de la que heredan las subclases. La clase Personajes es abstracta porque nunca se crea un personaje genérico: siempre será un Guerrero, un Mago o un Sacerdote. Lo mismo ocurre con Armas (de la que heredan ArmaCuerpoACuerpo y ArmaADistancia), con Hechizos (con cuatro subtipos) y con Estados (Quemadura, Veneno y Renovar). De este modo los atributos comunes (nombre, vida, daño base...) se definen una sola vez en la clase padre.

Polimorfismo

Las subclases sobrescriben métodos de la clase padre para comportarse de forma distinta. El Guerrero sobrescribe recibirDano para aplicar su probabilidad de bloqueo y la reducción por adrenalina; el Sacerdote sobrescribe curar para añadir su bonus de sanación; el Mago aporta calcularPoderHechizo, que multiplica su poder mágico. En las armas, cada subtipo implementa su propia versión de calcularDano, y cada estado define alAplicar, alProcesarTurno y alExpirar de forma específica.

Encapsulación y abstracción

Los atributos de cada clase se declaran protegidos o privados y se acceden mediante getters y setters, de modo que cada objeto controla su propio estado. Por ejemplo, el arma nunca modifica directamente la vida de un personaje: solo devuelve un número con calcularDano, y es el método recibirDano del personaje el que aplica la defensa y reduce la vida. Esta separación de responsabilidades mantiene las reglas del juego centralizadas y evita inconsistencias.

Estructuras de datos

El juego utiliza colecciones de forma natural. Cada equipo es un ArrayList<Personajes>; cada personaje guarda sus armas, hechizos y estados activos en listas; y los cooldowns de los hechizos se gestionan con un Map<String, Integer> donde la clave es el nombre del hechizo y el valor son los turnos que le quedan de recarga.

4.2. Materiales y métodos

Para el desarrollo del juego se utilizaron las siguientes herramientas y tecnologías:

- **Java (JDK):** lenguaje orientado a objetos en el que está programado todo el juego.
- **Eclipse IDE:** entorno de desarrollo donde se escribió, ejecutó y depuró el código.
- **JUnit:** framework de pruebas unitarias para validar la lógica de personajes, armas, hechizos y estados.
- **Apache Maven:** gestor de dependencias y de la compilación del proyecto (pom.xml).
- **Git:** control de versiones para llevar el seguimiento del código.

Metodología

El desarrollo fue incremental. Primero se diseñó la jerarquía de clases sobre el papel a partir de los requisitos. Después se implementaron las clases base abstractas (Personajes, Armas, Hechizos, Estados) y, a continuación, sus subclases. Una vez funcionaban las entidades por separado, se construyó el motor de combate (clase Combate) que las hace interactuar. Finalmente se añadieron las pruebas unitarias con JUnit y se refactorizó el código según las correcciones comentadas en clase, separando responsabilidades (modelo, lógica de combate y, más tarde, acceso a datos).

Estructura de paquetes

El código está organizado en paquetes según su responsabilidad:

Paquete	Contenido
personajes	Clase abstracta Personajes y subclases Guerrero, Mago, Sacerdote
armas	Clase abstracta Armas y subclases ArmaCuerpoACuerpo, ArmaADistancia
hechizos	Clase abstracta Hechizos y los cuatro subtipos de hechizo
estados	Clase abstracta Estados y subclases Quemadura, Veneno, Renovar
combate	Clase Combate: motor del juego por rondas
catalogo	CatalogoPersonajes: crea los 6 personajes con sus armas y hechizos
main	Main: menú, creación de equipos y bucle principal
db	Capa de persistencia (parte intermodular de Bases de Datos)

4.3. Resultados y análisis

El resultado es un juego funcional y jugable por consola. A continuación se describen las clases más relevantes con fragmentos del código real.

Jerarquía de personajes

La clase abstracta Personajes define los atributos y métodos comunes a todos los personajes. La vida y el recurso empiezan al máximo, y las colecciones (armas, estados, hechizos y cooldowns) nacen vacías:

```
public abstract class Personajes {
    protected int id;
    protected String nombre;
    protected int vidaMax, vidaActual;
    protected int recursoMax, recursoActual;
    protected int ataqueBase, defensaBase, poderMagico;
    protected Armas armaEquipada;
    protected ArrayList<Armas> armasDisponibles;
    protected ArrayList<Estados> estadosActivos;
    protected ArrayList<Hechizos> hechizos;
    protected Map<String, Integer> cooldowns;

    public Personajes(String nombre, int vidaMax, int recursoMax,
        int ataqueBase, int defensaBase, int poderMagico) {
        this.nombre = nombre;
        this.vidaMax = vidaMax;
        this.vidaActual = vidaMax;    // la vida empieza al maximo
        this.recursoMax = recursoMax;
        this.recursoActual = recursoMax;
        this.ataqueBase = ataqueBase;
        this.defensaBase = defensaBase;
        this.poderMagico = poderMagico;
        this.armasDisponibles = new ArrayList<>();
        this.estadosActivos = new ArrayList<>();
        this.hechizos = new ArrayList<>();
        this.cooldowns = new HashMap<>();
    }
}
```

Cada subclase añade sus atributos exclusivos y sobrescribe métodos para comportarse de forma distinta (polimorfismo). El Guerrero, por ejemplo, sobrescribe recibirDano para aplicar primero su probabilidad de bloqueo y luego la reducción por adrenalina antes de delegar en el método del padre:

```
public class Guerrero extends Personajes {
    private int defensaAdrenalina;
    private double probBloqueo;

    @Override
    public void recibirDano(int cantidadDano) {
        // Probabilidad de bloquear el ataque por completo
        if (Math.random() < this.probBloqueo) {
            System.out.println(this.nombre + " ¡ha bloqueado el
ataque!");
            return;
        }
        // La adrenalina reduce parte del daño antes de pasarlo al padre
        int danoReducido = cantidadDano - this.defensaAdrenalina;
        if (danoReducido < 0) danoReducido = 0;
        super.recibirDano(danoReducido);
    }
}
```

El Mago aporta el método calcularPoderHechizo, que escala su poder mágico, y el Sacerdote sobrescribe curar para añadir un bonus de sanación:

```
// Mago
public int calcularPoderHechizo() {
    return (int) (this.poderMagico * this.multiplicadorMagico);
}

// Sacerdote
@Override
public void curar(int cantidadVida) {
    int curacionFinal = cantidadVida + this.bonusSanacion;
    super.curar(curacionFinal);
}
```

Personajes del juego

El catálogo define los seis personajes con sus estadísticas, su clase, su atributo exclusivo y sus dos armas:

Personaje	Clase	Bando	Vida	Atrib. exclusivo
Geralt de Rivia	Guerrero	Bueno	800	DefAdrenalina 20 / Bloqueo 20%
Yennefer	Mago	Bueno	400	Mult. mágico x1.5
Ciri	Sacerdote	Bueno	600	Bonus sanación +20
Imlerith	Guerrero	Malo	1000	DefAdrenalina 30 / Bloqueo 15%
Caranthir	Mago	Malo	500	Mult. mágico x1.3
Eredin	Sacerdote	Malo	750	Bonus sanación +15

Armas: cálculo polimórfico del daño

La clase abstracta Armas obliga a cada subtipo a implementar calcularDano, que solo devuelve un número y nunca modifica la vida. ArmaCuerpoACuerpo aplica el modificador y comprueba el golpe crítico:

```
public abstract class Armas {
    protected String nombre, tipo;
    protected int danoBase;
    protected double modificador, probCritico;
    // Cada subclase calcula el daño a su manera
    public abstract int calcularDano(Personajes atacante, Personajes
defensor);
}

public class ArmaCuerpoACuerpo extends Armas {
    @Override
    public int calcularDano(Personajes atacante, Personajes defensor) {
        int danoTotal = (int) (this.danoBase * this.modificador);
        if (Math.random() < this.probCritico) {
            danoTotal = danoTotal * 2;    // golpe critico
        }
        return danoTotal;
    }
}
```

Hechizos y cooldowns

La clase Hechizos gestiona el coste de recurso y el cooldown a través del mapa que vive en el personaje. El método puedeUsarse comprueba que el hechizo no esté en recarga y que haya recurso suficiente antes de permitir lanzarlo:

```
public boolean puedeUsarse(Personajes lanzador) {
    int cdActual = lanzador.getCooldown(this.nombre);
    if (cdActual > 0) {
        System.out.println(" [COOLDOWN] " + nombre + " no
disponible.");
        return false;
    }
    return lanzador.gastarRecurso(this.costeMana);
}

public void ponerEnCooldown(Personajes lanzador) {
    lanzador.setCooldown(this.nombre, this.cooldownMaximo);
}
```

Existen cuatro tipos de hechizo, repartidos entre las clases: el Mago tiene daño directo y daño en el tiempo (Quemadura), el Sacerdote tiene curación directa y curación en el tiempo (Renovar), y el Guerrero usa habilidades físicas modeladas como hechizos con coste o cooldown (Golpe Poderoso, Embestida Brutal).

Estados alterados por turno

Los estados heredan de la clase abstracta Estados, que usa un enum (DOT para daño en el tiempo, HOT para curación en el tiempo) y define los métodos que cada subclase implementa. La clave del diseño es el método aplicarEstados, que evita acumular el mismo estado dos veces: si ya existe uno con el mismo nombre, renueva su duración; si no, lo añade:

```
public void aplicarEstados(Estados nuevoEstado) {
    for (Estados estado : estadosActivos) {
        if (estado.getNombre().equals(nuevoEstado.getNombre())) {
            // Ya existe: se renueva la duración en lugar de duplicarlo
            estado.renovarDuracion(nuevoEstado.getTurnosRestantes());
            return;
        }
    }
    // No existia: se añade y se ejecuta su efecto inicial
    estadosActivos.add(nuevoEstado);
    nuevoEstado.alAplicar(this);
}
```

Al final de cada ronda, procesarEstados recorre los estados activos, aplica su efecto, reduce su duración y elimina los que ya han expirado:

```
public void procesarEstados() {
    ArrayList<Estados> expirados = new ArrayList<>();
    for (Estados estado : estadosActivos) {
        estado.alProcesarTurno(this); // aplica daño o curación
        estado.reducirDuracion();
        if (!estado.estaActivo()) {
            estado.alExpirar(this);
            expirados.add(estado);
        }
    }
    estadosActivos.removeAll(expirados);
}
```

Los tres estados obligatorios son: Quemadura (DOT, resta vida cada turno con potencia alta y duración corta), Veneno (DOT, resta menos vida pero dura más turnos) y Renovar (HOT, cura vida cada turno sin superar la vida máxima).

Motor de combate por rondas

La clase Combate contiene el bucle principal del juego. Mientras ambos equipos tengan personajes vivos, cada ronda se divide en tres fases estrictas: las acciones de cada personaje vivo, el procesamiento de los estados de todos los personajes y la comprobación de la condición de victoria.

Fase	Qué ocurre
1. Acciones	Cada personaje vivo actúa: el jugador elige por menú y la IA decide automáticamente (objetivo aleatorio o el aliado más débil)

Fase	Qué ocurre
	para curar)
2. Procesamiento de estados	Se llama a procesarEstados() de cada personaje: se aplican daños y curaciones por turno y se eliminan los estados expirados
3. Comprobación de victoria	Si un equipo se queda sin personajes vivos, termina el combate y se muestra el resumen final

Si un personaje muere en mitad de la ronda, no llega a actuar cuando le toque. El combate termina inmediatamente en cuanto los tres personajes de un equipo quedan a 0 de vida, y entonces se muestra un resumen final con el equipo ganador, el número de rondas y el estado final de los personajes.

Salida por consola

Cada ronda imprime de forma detallada el estado de los personajes (vida, recurso y estados activos con sus turnos restantes), las acciones realizadas paso a paso y los efectos aplicados durante la fase de estados (daños, curaciones y expiraciones), tal como exige el enunciado.

5. DIAGRAMA DE CLASES

El siguiente diagrama de clases UML resume la arquitectura completa del juego y las relaciones de herencia entre sus entidades. Se aprecian las cuatro jerarquías principales: la clase abstracta Personajes con sus subclases Guerrero, Mago y Sacerdote; la clase abstracta Armas con ArmaCuerpoACuerpo y ArmaADistancia; la clase abstracta Hechizos con sus cuatro subtipos (DañoDirecto, DañoEnElTiempo, CuraciónDirecta y CuraciónEnElTiempo); y la clase abstracta Estados con Quemadura, Veneno y Renovar. Además se muestran las clases de apoyo Main, CatalogoPersonajes y Combate, que crean los equipos y coordinan el combate.

En la página siguiente se muestra el diagrama a tamaño completo para facilitar su lectura.

5.1. Diagrama de clases UML del juego

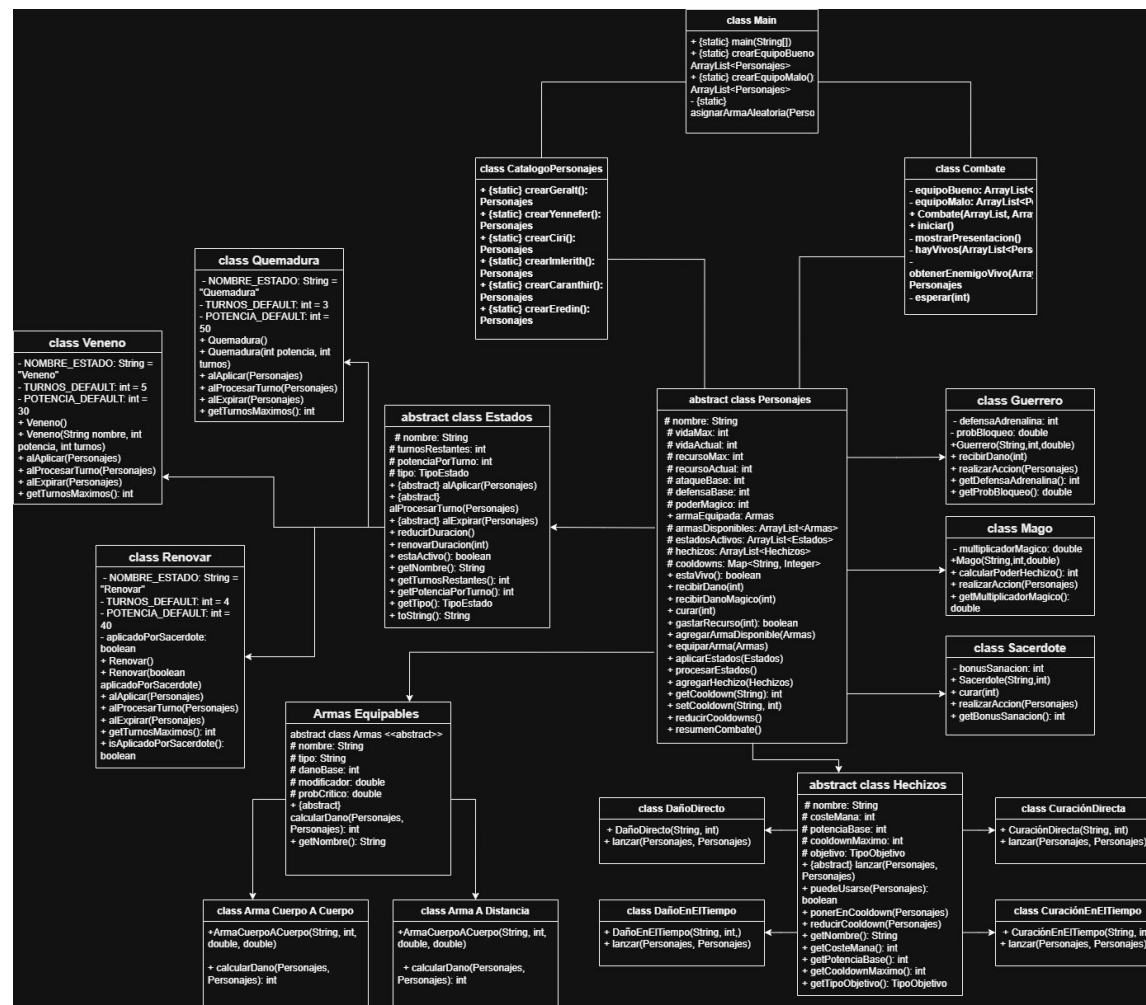


Figura 1. Diagrama de clases UML completo del juego Witcher RPG (jerarquías de personajes, armas, hechizos y estados).

6. CONCLUSIONES

El proyecto ha cumplido el objetivo de construir un juego de rol por turnos completo y funcional aplicando la programación orientada a objetos. La jerarquía de clases refleja con claridad las entidades del juego y permite que cada tipo de personaje, arma, hechizo y estado se comporte de forma distinta gracias al polimorfismo.

Entre los aprendizajes más importantes destaca el uso real de la herencia y el polimorfismo: una misma llamada a `recibirDano`, `calcularDano` o `alProcesarTurno` produce comportamientos diferentes según el objeto concreto, lo que evita largas cadenas de condicionales y hace el código más limpio y ampliable.

También se ha comprendido la utilidad de las colecciones (listas para equipos, hechizos y estados; un mapa para los cooldowns) y la importancia de separar responsabilidades: el arma calcula daño, el personaje gestiona su vida y el motor de combate coordina la interacción. Esta separación facilitó después añadir la capa de persistencia sin tocar la lógica del juego.

7. LÍNEAS DE INVESTIGACIÓN FUTURAS

A partir del juego actual, el proyecto puede ampliarse en varias direcciones:

- Orden de turnos basado en una estadística de velocidad en lugar de un orden fijo.
- Nuevas mecánicas: esquiva, resistencias elementales (fuego, hielo, sagrado) o hechizos de área que afecten a todo un equipo.
- Acciones adicionales en combate como Defender (reducir el daño hasta el siguiente turno) o Pasar turno.
- Un sistema de experiencia y niveles que recompense al jugador al terminar cada combate.
- Visualización de estadísticas del combate (daño y curación por personaje) mediante gráficos con la librería XChart.

8. BIBLIOGRAFÍA

Documentación oficial de Java (Oracle). <https://docs.oracle.com/en/java/>

Documentación de JUnit. <https://junit.org/junit5/docs/current/user-guide/>

Documentación de Apache Maven. <https://maven.apache.org/guides/>

Documentación de la librería XChart. <https://knowm.org/open-source/xchart/>

Apuntes y materiales de la asignatura de Programación, 1º DAM.

Documento de requisitos del proyecto proporcionado por el profesorado.